

S202: Ebenenlayout als Architekturhypothese

Paketordnung, lokale Platzierung und explizite Schnittkanten

Johannes Weigend

Entwurf, 24. Mai 2026

Zusammenfassung

S202 visualisiert Softwarearchitektur als geschichtetes Layout. Die entscheidende Aussage der Darstellung ist einfach: Ein Element, das ein anderes Element benutzt, steht oberhalb des benutzten Elements. In realen Java-Systemen sind Paketabhängigkeiten nicht direkt beobachtbar — sie entstehen implizit und können technisch nur aus den Klassenabhängigkeiten ermittelt werden. Paketordnung, lokale Platzierung und Zyklusbehandlung müssen daher getrennt berechnet werden. Dieses Dokument beschreibt das konzeptionelle Modell: Aus aggregierten Klassenabhängigkeiten entsteht eine Architekturhypothese für Pakete; konkrete Kanten, die dieser Ordnung widersprechen, werden als Backedges markiert; Kanten in SCCs werden für die Ordnungsberechnung geschnitten, bleiben aber als explizites Feedback sichtbar. Konsistenzregeln dienen anschliessend als implementierungsneutrale, ausführbare Spezifikation der erzeugten Grafik.

1 Motivation

Die geschichtete Darstellung von Abhängigkeiten ist ein etabliertes Mittel der Softwarevisualisierung. Die Grundidee ist einfach: Wer abhängig ist, steht oben; wer genutzt wird, steht unten.

S202 verfolgt dasselbe Ziel, aber mit einem anderen Anspruch. Die Darstellung soll dem Nutzer eine Architekturabstraktion liefern, die *ohne Pfeile auskommt* und trotzdem Klarheit über die Struktur gibt. Die vertikale Position eines Elements trägt die Information — nicht die einzelne Kante. Kanten sind in grossen Systemen nicht lesbar; eine stabile Schichtung ist es.

Das Ziel ist ausdrücklich die Visualisierung *grosser Anwendungen*. In realen Java-Systemen mit Tausenden von Klassen und Hunderten von Paketen ist eine manuelle Architekturzuweisung nicht praktikabel. S202 leitet die Schichtung automatisch aus den beobachteten Abhängigkeiten ab und macht dabei die zugrundeliegende Architekturhypothese explizit und nachvollziehbar.

2 Geltungsbereich

Dieses Dokument beschreibt das konzeptionelle Modell des geschichteten S202-Layouts. Es ist keine zeilengetreue Beschreibung der aktuellen Implementierung. Die Implementierung ist eine konkrete Ausprägung dieses Modells und kann angepasst werden, wo sie die konzeptionelle Struktur noch nicht präzise genug abbildet.

Der Text vermeidet deshalb absichtlich Details wie konkrete Schwellenwerte, historische Sonderfälle oder interne Namen einzelner Checks. Entscheidend ist die fachliche Aussage der Darstellung: Die Grafik soll erklären können, warum jedes sichtbare Element an seiner Position steht und warum jede Kante, die der Ordnung widerspricht, sichtbar bleibt.

3 Grundbegriffe

Eine Klassenabhängigkeit wird als

$$A \rightarrow B$$

geschrieben und bedeutet: Klasse oder Element A benutzt Klasse oder Element B . S202 ordnet Elemente auf Ebenen an. Kleinere Level liegen in der Grafik niedriger, grössere Level liegen höher.

Eine Abhängigkeit ist *ordnungskonform*, wenn der Nutzer oberhalb des benutzten Elements liegt:

$$A \rightarrow B \implies level(A) > level(B).$$

Diese Konvention gilt für Klassenlevel, Paketlevel und lokale UI-Level, aber die Berechnung erfolgt in zwei getrennten Schritten.

Zuerst entsteht die Paketordnung: Aus den aggregierten Klassenabhängigkeiten werden Paketgewichte berechnet. Daraus folgt eine stabile Anordnung der Pakete und ihrer Subpakete, die unabhängig von einzelnen Klassen ist.

Danach werden die Klassen eingeordnet: Innerhalb jedes Containers werden Klassen und sichtbare Subpakete anhand ihrer lokalen Abhängigkeiten untereinander sortiert. Die Paketordnung bleibt dabei erhalten — eine einzelne Klasse kann die Reihenfolge der Pakete nicht verschieben.

Eine Kante, die aus einem niedrigeren Element in ein höheres Element zeigt, widerspricht der berechneten Ordnung. Im Folgenden heisst eine solche Kante *gegenläufige Kante* oder *Backedge*. Sie ist kein Zeichenfehler: Sie ist ein Befund der Architekturhypothese.

4 Das Layoutversprechen

Abbildung 1 zeigt das Grundprinzip der S202-Darstellung. Links ist ein azyklisches Beispiel zu sehen. Die Kette $E \rightarrow A \rightarrow B \rightarrow D$ ist ordnungskonform:

$$L(E) = 3, \quad L(A) = 2, \quad L(B) = 1, \quad L(D) = 0.$$

Der Nutzer steht jeweils über dem benutzten Element. Die Abhängigkeit ist damit unmittelbar aus der vertikalen Ordnung lesbar.

Rechts ist ein zyklisches Beispiel zu sehen. Die gestrichelten Kanten sind keine zufälligen Layoutfehler. Sie sind ausgewählte Teilkanten von SCCs, die S202 für die Ebenenberechnung schneidet, um Zyklen aufzubrechen und dabei die Architekturhypothese zu erhalten. Die Abhängigkeit wird dadurch nicht gelöscht. Sie bleibt als explizite Schnittkante sichtbar.

Das Layoutversprechen lautet damit nicht: Die Grafik findet eine schöne Anordnung. Es lautet: Die Grafik macht die Architekturhypothese nachvollziehbar. Ordnungskonforme Abhängigkeiten passen zur Levelordnung. Kanten, die nicht zur Levelordnung passen, verschwinden nicht stillschweigend, sondern werden klassifiziert und sichtbar gemacht.

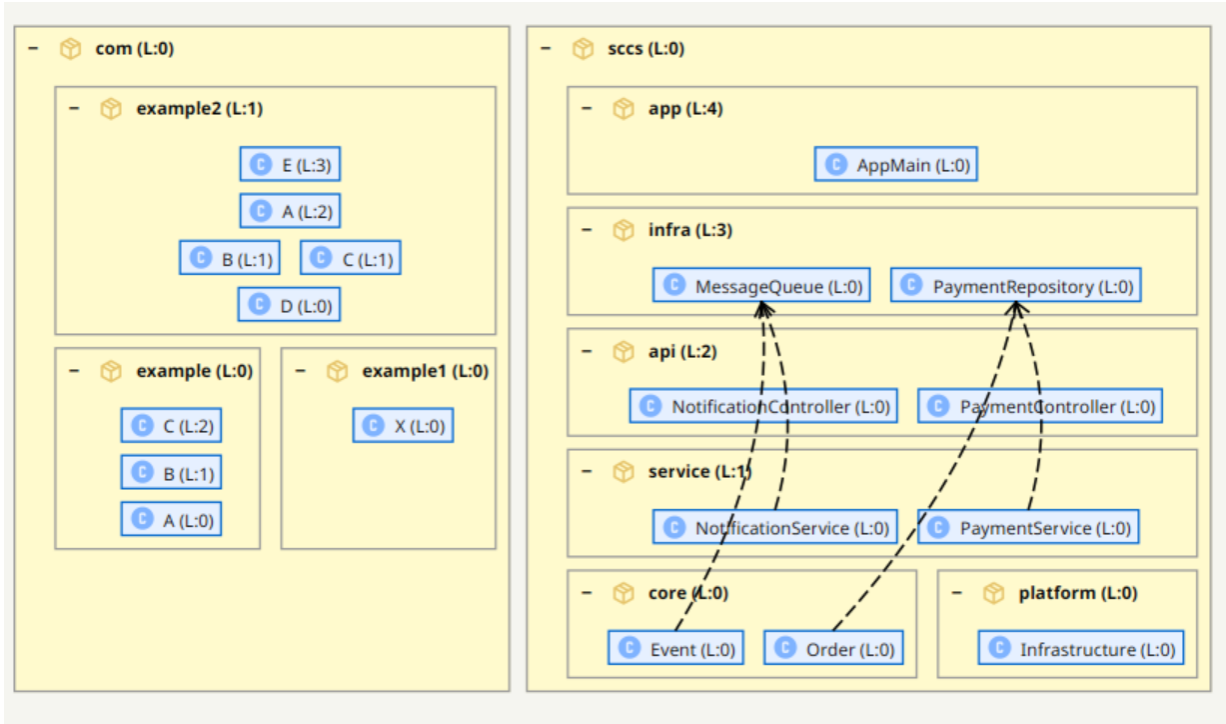


Abbildung 1: Das Layoutversprechen von S202. **Links:** ein azyklisches Beispiel. Levelwerte wachsen nach oben: D liegt auf Level 0, B und C auf Level 1, A auf Level 2 und E auf Level 3. Die Kette $E \rightarrow A \rightarrow B \rightarrow D$ ist ordnungskonform. **Rechts:** ein zyklisches Beispiel. Die gestrichelten Kanten sind geschnittene Teilkanten von SCCs. Sie werden aus der Ordnungsberechnung entfernt, bleiben aber als explizite Feedback-Kanten sichtbar.

5 Architekturhypothese aus Klassenabhängigkeiten

S202 beobachtet keine direkten Paketabhängigkeiten als Primärdaten. Das Werkzeug kennt zunächst nur Klassen und Klassenabhängigkeiten. Eine Paketbeziehung ist daher immer eine abgeleitete Aussage.

Für zwei Pakete P und Q summiert S202 die Abhängigkeiten von Klassen in P zu Klassen in Q . Dasselbe geschieht in der Gegenrichtung. Dadurch entstehen gewichtete Beziehungen:

$$w(P, Q) \quad \text{und} \quad w(Q, P).$$

Dominieren die Abhängigkeiten von P nach Q , dann interpretiert S202 P als Nutzerpaket und Q als tiefer liegendes, genutztes Paket:

$$w(P, Q) > w(Q, P) \implies level(P) > level(Q).$$

Diese Aussage ist eine Architekturhypothese, keine absolute Wahrheit. Sie sagt: Nach den beobachteten Klassenabhängigkeiten ist es plausibel, P oberhalb von Q einzuordnen. Die spätere lokale Platzierung darf diese Hypothese nicht unbemerkt umdeuten.

Der entscheidende konzeptionelle Schritt ist, dass S202 die Paketbeziehungen als *eigenständigen Graphen* behandelt — einen Paketbaum, der unabhängig von den einzelnen Klassen existiert. Dieser Baum entsteht aus den aggregierten Gewichten und liefert eine stabile Ordnung. Erst diese Stabilität macht das Schneideproblem in zyklischen Klassengraphen lösbar: Weil die Paketord-

nung feststeht, ist klar, welche Kanten gegen sie laufen — und genau diese werden als Kandidaten für den Schnitt betrachtet.

Sind die Gewichte in beiden Richtungen gleich groß,

$$w(P, Q) = w(Q, P),$$

dann liefert die Hypothese keine Ordnung zwischen P und Q . Beide Pakete sind echte Peers: Sie nutzen einander gleich stark, und keine Richtung dominiert. S202 lässt solche Paare auf demselben Level und schneidet keine Kante zwischen ihnen. Das Ergebnis ist ein gemeinsames SCC-Level, das die Symmetrie der Abhängigkeit ehrlich abbildet.

6 Zyklen und Schnittkanten

Zyklen sind der Grund, warum die Ebenenberechnung nicht als einfacher Durchlauf über alle Elemente funktioniert. In einem Zyklus können nicht alle Kanten gleichzeitig ordnungskonform sein. S202 muss daher entscheiden, welche Kanten für die Ordnungsberechnung ignoriert werden, ohne sie aus dem Modell oder aus der Grafik zu entfernen.

Ausgangspunkt ist die bereits berechnete Paketordnung. Eine konkrete Klassenabhängigkeit wird als Backedge markiert, wenn sie aus einem Paket kommt, das die Architekturhypothese niedriger eingeordnet hat, und in ein höher eingeordnetes Paket zeigt.

Ist eine solche Backedge Teil einer SCC, wird sie für die Ordnungsberechnung geschnitten. Das Schneidekriterium ist dabei präzise: Geschnitten wird die Kante mit dem grössten Levelunterschied zwischen Quell- und Zielpaket — also die Kante, die am stärksten gegen die Architekturhypothese läuft. Abbildung 2 zeigt ein Beispiel: Die Kante `core` → `api` überspringt zwei Level (von 0 nach 2) und ist damit die eindeutige Schnittkante. Eine Kante `core` → `service` hätte nur einen Level übersprungen und wäre nachrangig.

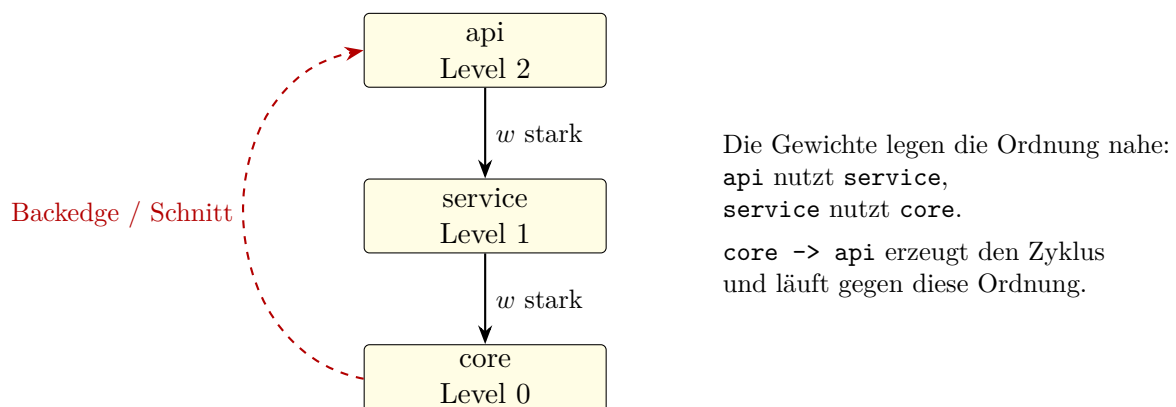


Abbildung 2: Schematische Schnittregel. Die Paketordnung entsteht aus aggregierten Klassenabhängigkeiten. Eine konkrete Abhängigkeit aus `core` nach `api` läuft von Level 0 nach Level 2 und damit gegen die Architekturhypothese. Ist diese Kante Teil einer SCC, wird sie für die Ordnungsberechnung geschnitten und bleibt als Schnittkante sichtbar.

Der Schnitt hat eine enge Bedeutung: Er entfernt die Kante nur aus dem Graphen, auf dem die Level berechnet werden. Die reale Abhängigkeit bleibt erhalten. Gerade dadurch wird die Darstellung ehrlich: Der Zyklus wird für die Berechnung aufgebrochen, aber seine Ursache bleibt sichtbar.

7 Lokale Platzierung

Die Paketordnung beantwortet die Frage, welche Pakete nach der Architekturhypothese oberhalb oder unterhalb anderer Pakete liegen. Sie beantwortet aber nicht automatisch, wo eine Klasse innerhalb ihres sichtbaren Elternpakets stehen soll.

Dafür verwendet S202 eine lokale Platzierung. Innerhalb eines Containers werden nur die direkten Geschwister betrachtet: Klassen und sichtbare Subpakete, die um eine Position im selben Elternpaket konkurrieren. Aus den Klassenabhängigkeiten zwischen diesen Geschwistern entsteht ein lokaler Graph. Auf diesem Graphen werden lokale UI-Level berechnet.

Die lokale Platzierung darf die Paketarchitektur nicht neu definieren. Wenn die Architekturhypothese ein Subpaket unterhalb eines anderen Containers sieht, darf ein lokaler Sortierschritt diese Aussage nicht stillschweigend umkehren, nur weil dadurch einige Linien kürzer oder weniger gegenläufig aussehen. Layout ist hier der Architekturhypothese untergeordnet.

Das ist der Grund für die Trennung der Begriffe:

- Klassenlevel beschreiben die Ordnung im Klassenabhängigkeitsgraphen.
- Paketlevel beschreiben die aus Klassenabhängigkeiten aggregierte Architekturhypothese für Pakete.
- Lokale UI-Level beschreiben die sichtbare Position innerhalb eines konkreten Containers.

Alle drei verwenden dieselbe Grundidee: Nutzer stehen höher als das genutzte Element. Sie dürfen aber nicht zu einem einzigen globalen Level vermischt werden.

8 Konsistenzregeln

Konsistenzregeln waren in der Entwicklung von S202 nicht nur Testcode. Sie waren die implementierungsneutrale Formulierung dessen, was eine konsistente Grafik bedeuten muss.

Der Layoutalgorithmus erzeugt Modell, Level, Kantenklassifikationen und sichtbare Reihenfolge. Die Konsistenzregeln werden danach geprüft. Sie schneiden keine Kanten, verschieben keine Elemente und optimieren kein Layout. Sie prüfen, ob die fertige Grafik ihre eigene Architekturhypothese einhält.

Der Vorteil dieser Formulierung liegt in der Redundanz. Eine Konsistenzregel muss nicht wissen, mit welchen internen Schritten der Layoutalgorithmus zu seinem Ergebnis gekommen ist. Sie kann unabhängig implementiert werden und nur das fertige Ergebnis lesen. Dadurch prüft ein zweiter Codepfad dieselbe fachliche Aussage.

Konzeptionell reichen drei Konsistenzregeln:

1. Ordnungskonforme Abhängigkeiten erfüllen $level(A) > level(B)$.
2. Eine nicht ordnungskonforme Kante verschwindet nicht. Sie ist als Backedge, SCC-Kante, Schnittkante oder Verletzung klassifiziert.
3. Die sichtbare UI-Ordnung stimmt mit dem Modell überein: Die Paketordnung aus der Architekturhypothese und die lokale Einordnung der Klassen innerhalb ihrer Container sind konsistent dargestellt.

Die aktuelle Implementierung zerlegt diese Konsistenzregeln in vier konkrete Checks:

- R1** Prüft Konsistenzregel 1 für Klassenabhängigkeiten: Jede nicht als Backedge markierte Kante $A \rightarrow B$ muss $level(A) > level(B)$ erfüllen.
- R2** Prüft Konsistenzregel 1 für Pakete in derselben SCC: Peers ohne dominante Abhängigkeitsrichtung erhalten denselben Level.
- R3** Prüft Konsistenzregel 1 für Paketabhängigkeiten: Für jede gewichtete Paketkante mit $w(P, Q) > w(Q, P)$ muss $level(P) > level(Q)$ gelten.
- R4** Prüft Konsistenzregel 2: Jede Abhängigkeit trägt eine Klassifikation (NORMAL, VIOLATION, INTRA_SCC). Kanten ohne Klassifikation sind ein Implementierungsfehler.

Konsistenzregel 3 (Level-Tupel-Vergleich bei verschachtelten UI-Elementen) wird durch R1 und R3 in Teilen abgedeckt, ist aber noch kein vollständiger maschineller Check. Sie gilt konzeptionell als Ziel der Darstellung.

9 Umsetzung in S202

Aus dem konzeptionellen Modell ergibt sich eine klare Pipeline:

1. Klassenabhängigkeiten aus dem analysierten Code lesen.
2. Paketgewichte aus Klassenabhängigkeiten aggregieren.
3. Aus den Paketgewichten eine Architekturhypothese berechnen (Paketordnung).
4. Klassenlevel bestimmen: Zyklen im Klassengraphen werden entlang der Architekturhypothese geschnitten. Eine Kante $A \rightarrow B$ in einem Zyklus ist eine Schnittkante, wenn $pkgLevel(A) < pkgLevel(B)$. Für Zyklen innerhalb gleich-levelliger Pakete greift die Grad-Heuristik als Fallback.
5. Kanten, die nach Schritt 4 gegen die berechnete Ordnung laufen, als Backedges klassifizieren und sichtbar halten.
6. Lokale UI-Level pro sichtbarem Container berechnen.
7. Die fertige Grafik durch Konsistenzregeln prüfen (R1–R4).

10 Zusammenfassung

S202 macht die Architekturhypothese sichtbar: Wer wen benutzt, bestimmt die vertikale Position — nicht Kriterien wie kurze Kanten oder wenige Kreuzungen. Diese Hypothese entsteht aus Klassenabhängigkeiten, die zu Paketgewichten aggregiert werden. Daraus folgt eine Paketordnung. Konkrete Kanten, die gegen diese Ordnung laufen, werden als Backedges sichtbar. Sind sie Teil einer SCC, werden sie für die Ordnungsberechnung geschnitten, aber nicht aus dem Modell entfernt.

Die zentrale Qualität der Darstellung ist damit nicht, dass alle Linien unauffällig aussehen. Die zentrale Qualität ist Nachvollziehbarkeit: Jede sichtbare Abweichung von der Ordnung muss erklärt werden können.